

Optimización de la permitividad eléctrica relativa en una guía de onda con ocho slabs dieléctricos

V. Rodríguez¹

¹ Chapultepec 78295, Facultad de Ciencias, Universidad Autónoma de San Luis Potosí San Luis Potosí, S.L.P.

E-mail: *A310390@alumnos.uaslp.mx

28 Junio, 2024

SUMMARY: El propósito de este escrito es reportar los resultados experimentales para los coeficientes de magnitud lineal, tanto de la reflexión como de la transmisión de ondas electromagnéticas medidos por un analizador vectorial de redes. Una vez obtenidas las mediciones, se realizó un algoritmo simple de optimización con el objetivo de caracterizar el comportamiento de la onda en la guía de onda en presencia de 8 placas de FR4 y determinar el valor de permitividad eléctrica relativa que mejor se aproxima a los datos medidos experimentalmente, basado en cálculos teóricos realizados por medio del método de la matriz de transferencia.

1. MONTAJE EXPERIMENTAL

Se introdujeron 8 placas de FR4 (con un grosor de $d = 1.55$ [mm]) no espaciadas igualmente dentro de una guía de onda rectangular "WR90", las cuales estaban sostenidas por pequeñas estructuras fabricadas de PLA. Posteriormente, se conectaron ambos puertos de un analizador vectorial de redes a los extremos de la guía de onda y se realizó una barrida de frecuencias desde 1 GHz hasta 12 GHz para encontrar los parámetros "S" de la configuración.

Cada espaciamiento entre las placas de FR4 está caracterizado por una distancia " l_i ". Estas distancias fueron ordenadas de la siguiente manera: $l_7, l_6, \dots, l_1$.

por medio de un producto de matrices de la siguiente forma:

$$M = M_{\text{slab}} M_{l_7} M_{\text{slab}} \cdots M_{l_2} M_{\text{slab}} M_{l_1} M_{\text{slab}} \quad (1)$$

Donde M_{slab} representa la matriz de transferencia para una placa dieléctrica y M_{l_i} representa la propagación de la onda en el vacío durante una distancia l_i .

Los cálculos detallados para determinar las entradas de la matriz de transferencia para placas dieléctricas y para el espacio vacío se derivan del trabajo de (1):

Las entradas pertenecientes a la matriz M_{slab} son las siguientes:

Table 1: Medidas del espaciamiento que hay entre placas de FR4

l_i	Medida [mm]
l_1	9.29
l_2	8.95
l_3	8.63
l_4	9.11
l_5	10.46
l_6	9.56
l_7	9.04

$$\begin{aligned} M_{11} &= \cos(qd) + i \frac{1}{2} \left(\frac{q}{k} + \frac{k}{q} \right) \sin(qd) \\ M_{22} &= \cos(qd) - i \frac{1}{2} \left(\frac{q}{k} + \frac{k}{q} \right) \sin(qd) \\ M_{12} &= i \frac{1}{2} \left(\frac{q}{k} - \frac{k}{q} \right) \\ M_{21} &= -i \frac{1}{2} \left(\frac{q}{k} - \frac{k}{q} \right) \end{aligned} \quad (2)$$

Donde las constantes son:

$$\begin{aligned} \epsilon &= \epsilon' + i\epsilon'' \\ k &= \frac{2\pi}{c} \sqrt{v^2 - \left(\frac{c}{2w} \right)^2} \\ q &= \frac{2\pi}{c} \sqrt{\epsilon v^2 - \left(\frac{c}{2w} \right)^2} \end{aligned} \quad (3)$$

2. DESARROLLO TEORICO

Para modelar el comportamiento de las ondas electromagnéticas a través de las placas de FR4, se aplicó el método de la matriz de transferencia. Esta teoría nos permite describir la interacción de las ondas electromagnéticas con medios que están en capas

La matriz de transferencia para el espacio libre es diagonal y depende de l_i :

$$M(l_i) = \text{diag}(e^{ikl_i}, e^{-ikl_i}), \quad (4)$$

Una vez calculado el producto de matrices será posible encontrar los coeficientes R y T de la siguiente manera:

$$R = \frac{|M_{21}|^2}{|M_{22}|^2} \quad (5)$$

$$T = \frac{1}{|M_{22}|^2}$$

Con estos dos coeficientes es posible calcular la absorción, que está definida como:

$$\text{Absorción} = 1 - R - T \quad (6)$$

3. RESULTADOS EXPERIMENTALES

A continuación, se presentan las gráficas obtenidas mediante el uso del VNA, que muestran la magnitud lineal de la reflexión y la transmisión.

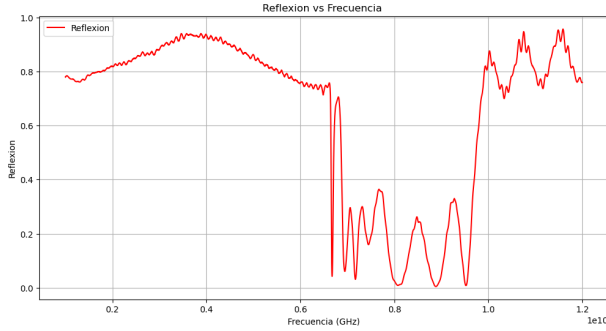


Fig. 1: Frecuencia vs Reflexión

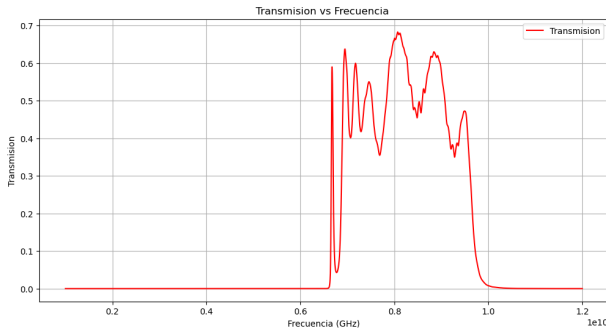


Fig. 2: Frecuencia vs Transmisión

4. ANÁLISIS DE RESULTADOS

Se desarrolló un código en Python para realizar los cálculos necesarios y determinar teóricamente los elementos de matriz definidos en la ecuación ((2)). Además, se graficaron los coeficientes R y T para

compararlos con las gráficas obtenidas experimentalmente.

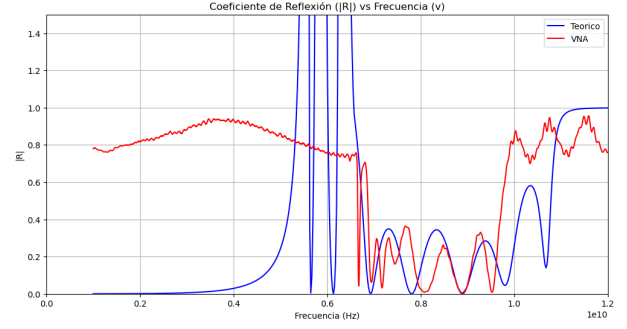


Fig. 3: Frecuencia vs Reflexión

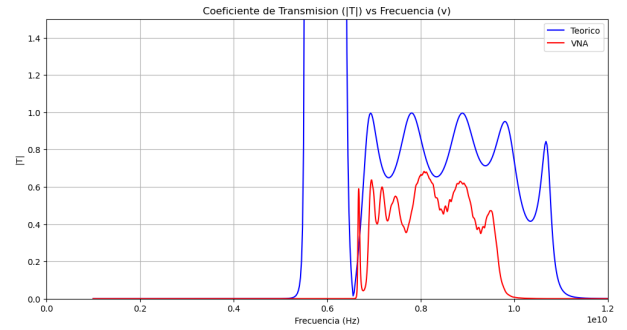


Fig. 4: Frecuencia vs Transmisión

El cálculo teórico toma frecuencias desde 1 GHz hasta 12 GHz. Sin embargo, como se puede observar, la predicción no tiene sentido previo a la frecuencia de corte definida en nuestro caso como 6.6 GHz. Por lo tanto, los análisis próximos se realizarán tomando en cuenta únicamente las frecuencias posteriores a esta frecuencia de corte.

4.1. Ajuste

Los únicos parámetros libres para ser variados son ϵ' y ϵ'' . Sin embargo, es muy poco eficiente probar a mano distintos valores. Por lo tanto, se implementó un código que realice ese trabajo. Para ello, definimos la variable error para 3 casos distintos:

(i) Respecto a la Reflexión:

$$\text{error}_R = \sum_i (R_{\text{teórico}}(v_i) - R_{\text{VNA}}(v_i))^2 \quad (i)$$

(ii) Respecto a la Transmisión:

$$\text{error}_T = \sum_i (T_{\text{teórico}}(v_i) - T_{\text{VNA}}(v_i))^2 \quad (ii)$$

(iii) Respecto a la suma:

$$\text{error}_s = \text{error}_R + \text{error}_T \quad (iii)$$

4.2. Optimización

Para encontrar los valores más óptimos de ϵ , se inicia con los “valores de prueba” $\epsilon' = 1$ y $\epsilon'' = 0.001$. Estos valores se ingresan al algoritmo que calcula los coeficientes R y T , y finalmente se calcula el error (ver ecuaciones (i), (ii) y (iii), dependiendo del caso). Se empleó la extensión “minimize” de la librería “Scipy” para desarrollar el algoritmo y encontrar los coeficientes de ϵ que minimizan el error.

(i) Respecto a la reflexión:

El proceso de minimización de error arrojó como resultado $\epsilon = 4.357416479873762 - 0.02001809506508289i$, con un $error_R = 4.901785989502279$, mostrando las siguientes gráficas:

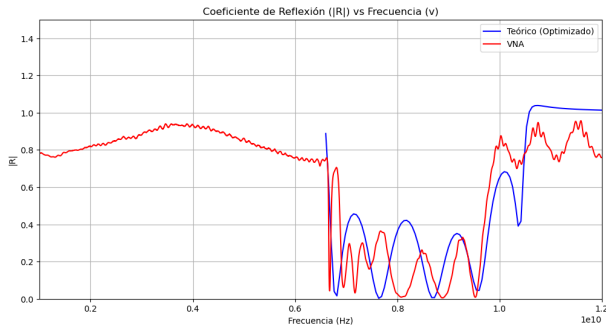


Fig. 5: Frecuencia vs Reflexión (optimizada respecto a R)

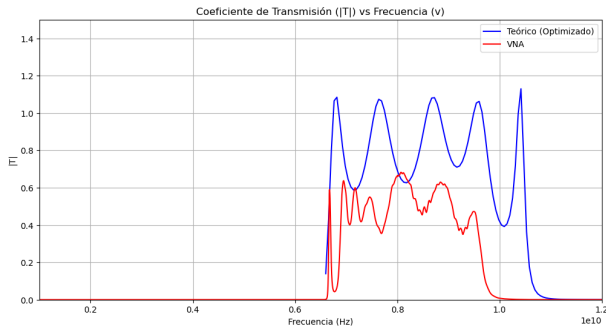


Fig. 6: Frecuencia vs Transmisión (optimizada respecto a R)

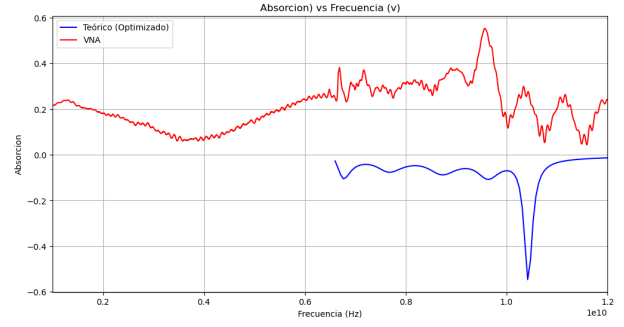


Fig. 7: Frecuencia vs Absorción (optimizada respecto a R)

(ii) Respecto a la Transmisión:

El proceso de minimización de error arrojó como resultado $\epsilon = 5.546125393915104 + 0.06630914116904014i$, con un $error_T = 0.7898738933746645$, mostrando las siguientes gráficas:

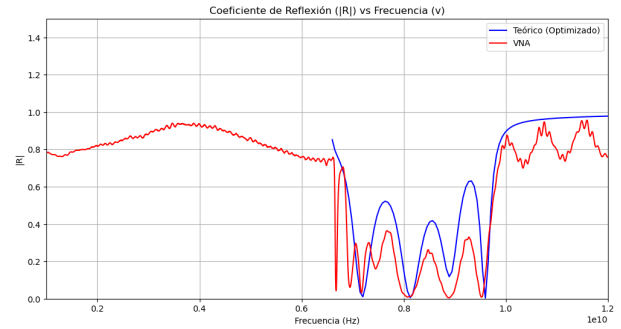


Fig. 8: Frecuencia vs Reflexión (optimizada respecto a T)

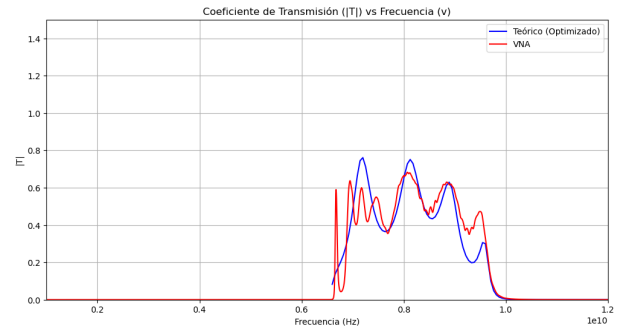


Fig. 9: Frecuencia vs Transmisión (optimizada respecto a T)

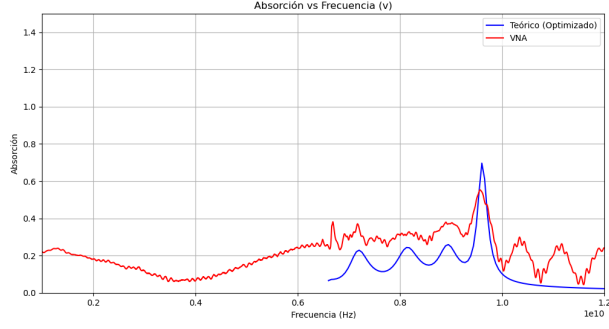


Fig. 10: Frecuencia vs Absorción (optimizada respecto a T)

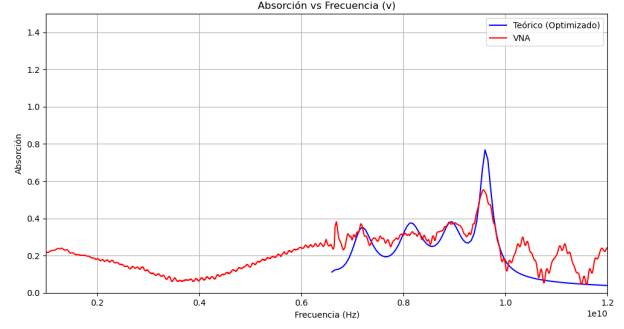


Fig. 13: Frecuencia vs Absorción (optimizada respecto a suma)

(iii) Respecto a la suma:

El proceso de minimización de error arrojó como resultado $\epsilon = 5.543149522313741 + 0.11847355818800968i$, con un $error_s = 3.58989019771032955$, mostrando las siguientes gráficas:

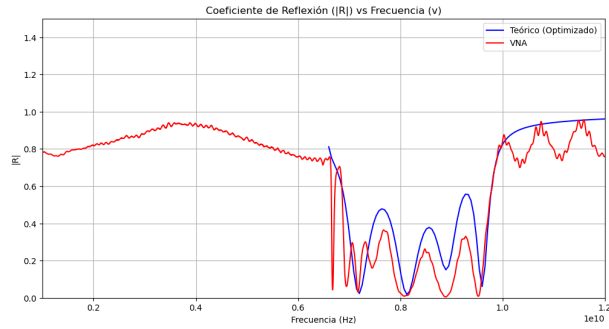


Fig. 11: Frecuencia vs Reflexión (optimizada respecto a suma)

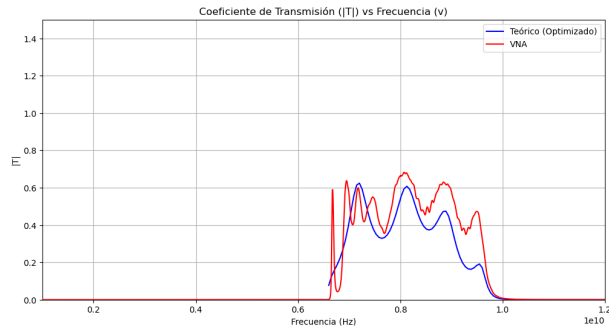


Fig. 12: Frecuencia vs Transmisión (optimizada respecto a suma)

El código para cada caso se puede revisar en los Apéndices A, B y C. Se intentó realizar una optimización con base en la absorción pero el resultado fue tan malo que dejó de ser considerado una opción.

5. CONCLUSIÓN

En base al análisis de los resultados, es fácil notar que la mejor aproximación a los datos experimentales se obtiene optimizando el valor de ϵ con respecto a la variable T .

Table 2: Error calculado

error	valor
$error_R$	4.9017
$error_T$	0.7898
$error_s$	3.5898

Por lo que la permitividad eléctrica relativa que mejor caracteriza a la guía de onda con los 8 slabs corresponde a:

$$\epsilon = 5.546125393915104 + 0.06630914116904014i \quad (7)$$

REFERENCES

- [1] A. A. Fernández-Marín, C. A. Flores-Castro, E. Ramírez-Hintze, V. Domínguez-Rocha, and J. A. Franco-Villafañe, "Tunable Perfect Absorption of Microwave Radiation via Dielectric Slabs in Irregular Arrangements: A Non-Hermitian Approach," .

A. Código en Python para R

A continuación se muestra el código en Python utilizado para los cálculos de minimización de error respecto a R :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Ruta al archivo de texto
file_path = r"C:\Users\rodri\Downloads\8 SLABS modificado.txt"

# Leer el archivo de texto con el delimitador de tabulación
data = pd.read_csv(file_path, delimiter='\\t')

# Extraer las frecuencias y magnitudes
freq1 = data['FREQ1.GHZ'] * 1e9
linmag1 = data['LINMAG1'] ** 2
linmag2 = data['LINMAG2'] ** 2

# Coeficientes
d = 0.00155
w = 0.02286 #real
c = 3e8
#l_val = [9.29e-3, 8.95e-3, 8.96e-3, 9.11e-3, 10.46e-3, 9.56e-3, 9.04e-3] #como en el papel
l_val = [9.04e-3, 9.56e-3, 10.46e-3, 9.11e-3, 8.96e-3, 8.95e-3, 9.29e-3]
# Vectores
v_val = np.linspace(6.6e9, 12e9, 100)

def calcular_RT(e_real, e_imag):
    e = e_real + e_imag * 1j
    R_values = np.zeros(len(v_val))
    T_values = np.zeros(len(v_val))
    for i in range(len(v_val)):
        v = v_val[i]
        k = 2 * np.pi * (np.sqrt((v**2 - (c / (2 * w))**2) + 0 * 1j)) / c
        q = 2 * np.pi * (np.sqrt((e * v**2 - (c / (2 * w))**2) + 0 * 1j)) / c

        # Elementos de matriz M_slab
        M11 = np.cos(q * d) + 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M22 = np.cos(q * d) - 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M12 = 1j / 2 * ((q / k - k / q) * np.sin(q * d))
        M21 = -1j / 2 * ((q / k - k / q) * np.sin(q * d))

        M_slab = np.array([[M11, M12], [M21, M22]])

    # Inicialización de la matriz total
    M_Total = np.eye(2, dtype=complex)

    # Elementos de matriz M_vacio y multiplicación de matrices
    for p in range(7):
        M_Total = np.matmul(M_Total, M_slab)
        l = l_val[p]
        M_vacio = np.array([[np.exp(1j * k * l), 0], [0, np.exp(-1j * k * l)]])
        M_Total = np.matmul(M_Total, M_vacio)

    M_Total = np.matmul(M_Total, M_slab)
```

```

        # Calculo coeficientes de R y T
        R = np.abs(M_Total[1, 0])**2 / np.abs(M_Total[1, 1])**2
        T = 1 / np.abs(M_Total[1, 1])**2

        R_values[i] = R
        T_values[i] = T
    return R_values, T_values

def error(params):
    e_real, e_imag = params
    R_teorico, _ = calcular_RT(e_real, e_imag)
    # Seleccionar solo los datos a partir de 6.6 GHz
    freq_mask = freq1 >= 6.6e9
    freq1_filtered = freq1[freq_mask]
    linmag1_filtered = linmag1[freq_mask]
    v_val_filtered = v_val
    R_teorico_filtered = R_teorico
    error = np.sum((R_teorico_filtered - np.interp(v_val_filtered, freq1_filtered, linmag1_filtered)))
    return error

# Valores iniciales para epsilon
initial_guess = [1, 0.001]

# Optimización
result = minimize(error, initial_guess, method='Nelder-Mead')
e_opt_real, e_opt_imag = result.x
e_opt = e_opt_real + e_opt_imag * 1j
opt_error = result.fun

# Calcular reflexión y transmisión con epsilon optimizado
R_opt, T_opt = calcular_RT(e_opt_real, e_opt_imag)

#Calcular suma
suma = R_opt + T_opt

#Calcular absorcion
absorcion = 1 - R_opt - T_opt
absorcionVNA = 1 - linmag1 - linmag2
# Graficar R vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, R_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag1, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|R|')
plt.title('Coeficiente de Reflexión (|R|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar T vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, T_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag2, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|T|')
plt.title('Coeficiente de Transmisión (|T|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)

```

```
plt.legend()

plt.show()

# Graficar Absorcion vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, absorcion, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, absorcionVNA, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Absorcion')
plt.title('Absorcion vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
#plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar Suma vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, suma, color='b', label='Teórico (Optimizado)')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Suma')
plt.title('Suma vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

plt.show()

print(f"Valor optimizado de epsilon: {e_opt}")
print(f"Error optimizado: {opt_error}")
```

B. Código en Python para T

B continuación se muestra el código en Python utilizado para los cálculos de minimización de error respecto a T :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Ruta al archivo de texto
file_path = r"C:\Users\rodri\Downloads\8 SLABS modificado.txt"

# Leer el archivo de texto con el delimitador de tabulación
data = pd.read_csv(file_path, delimiter='\t')

# Extraer las frecuencias y magnitudes
freq1 = data['FREQ1.GHZ'] * 1e9
linmag1 = data['LINMAG1'] ** 2
linmag2 = data['LINMAG2'] ** 2

# Coeficientes
d = 0.00155
w = 0.02286 #real
c = 3e8
#l_val = [9.29e-3, 8.95e-3, 8.96e-3, 9.11e-3, 10.46e-3, 9.56e-3, 9.04e-3] #como en el papel
l_val = [9.04e-3, 9.56e-3, 10.46e-3, 9.11e-3, 8.96e-3, 8.95e-3, 9.29e-3]
# Vectores
v_val = np.linspace(6.6e9, 12e9, 100)

def calcular_RT(e_real, e_imag):
    e = e_real + e_imag * 1j
    R_values = np.zeros(len(v_val))
    T_values = np.zeros(len(v_val))
    for i in range(len(v_val)):
        v = v_val[i]
        k = 2 * np.pi * (np.sqrt((v**2 - (c / (2 * w))**2) + 0 * 1j)) / c
        q = 2 * np.pi * (np.sqrt((e * v**2 - (c / (2 * w))**2) + 0 * 1j)) / c

        # Elementos de matriz M_slab
        M11 = np.cos(q * d) + 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M22 = np.cos(q * d) - 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M12 = 1j / 2 * ((q / k - k / q) * np.sin(q * d))
        M21 = -1j / 2 * ((q / k - k / q) * np.sin(q * d))

        M_slab = np.array([[M11, M12], [M21, M22]])

    # Inicialización de la matriz total
    M_Total = np.eye(2, dtype=complex)

    # Elementos de matriz M_vacio y multiplicación de matrices
    for p in range(7):
        M_Total = np.matmul(M_Total, M_slab)
        l = l_val[p]
        M_vacio = np.array([[np.exp(1j * k * l), 0], [0, np.exp(-1j * k * l)]])
        M_Total = np.matmul(M_Total, M_vacio)

    M_Total = np.matmul(M_Total, M_slab)
    # Calculo coeficientes de R y T
    R = np.abs(M_Total[1, 0])**2 / np.abs(M_Total[1, 1])**2
```

```

        T = 1 / np.abs(M_Total[1, 1])**2

        R_values[i] = R
        T_values[i] = T
    return R_values, T_values

def error(params):
    e_real, e_imag = params
    _, T_teorico = calcular_RT(e_real, e_imag)
    # Seleccionar solo los datos a partir de 6.6 GHz
    freq_mask = freq1 >= 6.6e9
    freq1_filtered = freq1[freq_mask]
    linmag2_filtered = linmag2[freq_mask]
    v_val_filtered = v_val
    T_teorico_filtered = T_teorico
    error = np.sum((T_teorico_filtered - np.interp(v_val_filtered, freq1_filtered, linmag2_filtered)))
    return error

# Valores iniciales para epsilon
initial_guess = [1, 0.001]

# Optimización
result = minimize(error, initial_guess, method='Nelder-Mead')
e_opt_real, e_opt_imag = result.x
e_opt = e_opt_real + e_opt_imag * 1j
opt_error = result.fun

# Calcular reflexión y transmisión con epsilon optimizado
R_opt, T_opt = calcular_RT(e_opt_real, e_opt_imag)

#Calcular suma
suma = R_opt + T_opt

#Calcular absorcion
absorcion = 1 - R_opt - T_opt
absorcionVNA = 1 - linmag1 - linmag2
# Graficar R vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, R_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag1, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|R|')
plt.title('Coeficiente de Reflexión (|R|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar T vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, T_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag2, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|T|')
plt.title('Coeficiente de Transmisión (|T|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

```

```
plt.show()

# Graficar Absorción vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, absorcion, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, absorcionVNA, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Absorción')
plt.title('Absorción vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar Suma vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, suma, color='b', label='Teórico (Optimizado)')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Suma')
plt.title('Suma vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

plt.show()

print(f"Valor optimizado de epsilon: {e_opt}")
print(f"Error optimizado: {opt_error}")
```

C. Código en Python para *suma*

A continuación se muestra el código en Python utilizado para los cálculos de minimización de error respecto a *suma*:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

# Ruta al archivo de texto
file_path = r"C:\Users\rodri\Downloads\8 SLABS modificado.txt"

# Leer el archivo de texto con el delimitador de tabulación
data = pd.read_csv(file_path, delimiter='\t')

# Extraer las frecuencias y magnitudes
freq1 = data['FREQ1.GHZ'] * 1e9
linmag1 = data['LINMAG1'] ** 2
linmag2 = data['LINMAG2'] ** 2

# Coeficientes
d = 0.00155
w = 0.02286 #real
c = 3e8
#l_val = [9.29e-3, 8.95e-3, 8.96e-3, 9.11e-3, 10.46e-3, 9.56e-3, 9.04e-3] #como en el papel
l_val = [9.04e-3, 9.56e-3, 10.46e-3, 9.11e-3, 8.96e-3, 8.95e-3, 9.29e-3]
# Vectores
v_val = np.linspace(6.6e9, 12e9, 100)

def calcular_RT(e_real, e_imag):
    e = e_real + e_imag * 1j
    R_values = np.zeros(len(v_val))
    T_values = np.zeros(len(v_val))
    for i in range(len(v_val)):
        v = v_val[i]
        k = 2 * np.pi * (np.sqrt((v**2 - (c / (2 * w))**2) + 0 * 1j)) / c
        q = 2 * np.pi * (np.sqrt((e * v**2 - (c / (2 * w))**2) + 0 * 1j)) / c

        # Elementos de matriz M_slab
        M11 = np.cos(q * d) + 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M22 = np.cos(q * d) - 1j / 2 * (q / k + k / q) * np.sin(q * d)
        M12 = 1j / 2 * ((q / k - k / q) * np.sin(q * d))
        M21 = -1j / 2 * ((q / k - k / q) * np.sin(q * d))

        M_slab = np.array([[M11, M12], [M21, M22]])

    # Inicialización de la matriz total
    M_Total = np.eye(2, dtype=complex)

    # Elementos de matriz M_vacio y multiplicación de matrices
    for p in range(7):
        M_Total = np.matmul(M_Total, M_slab)
        l = l_val[p]
        M_vacio = np.array([[np.exp(1j * k * l), 0], [0, np.exp(-1j * k * l)]])
        M_Total = np.matmul(M_Total, M_vacio)

    M_Total = np.matmul(M_Total, M_slab)
    # Calculo coeficientes de R y T
    R = np.abs(M_Total[1, 0])**2 / np.abs(M_Total[1, 1])**2
    T = 1 / np.abs(M_Total[1, 1])**2
```

```

        R_values[i] = R
        T_values[i] = T
    return R_values, T_values

def error(params):
    e_real, e_imag = params
    R_teorico, T_teorico = calcular_RT(e_real, e_imag)
    # Seleccionar solo los datos a partir de 6.6 GHz
    freq_mask = freq1 >= 6.6e9
    freq1_filtered = freq1[freq_mask]
    linmag1_filtered = linmag1[freq_mask]
    linmag2_filtered = linmag2[freq_mask]
    v_val_filtered = v_val
    R_teorico_filtered = R_teorico
    T_teorico_filtered = T_teorico
    error_R = np.sum((R_teorico_filtered - np.interp(v_val_filtered, freq1_filtered, linmag1_filtered)
    error_T = np.sum((T_teorico_filtered - np.interp(v_val_filtered, freq1_filtered, linmag2_filtered)
    return error_R + error_T

# Valores iniciales para epsilon
initial_guess = [1, 0.001]

# Optimización
result = minimize(error, initial_guess, method='Nelder-Mead')
e_opt_real, e_opt_imag = result.x
e_opt = e_opt_real + e_opt_imag * 1j
opt_error = result.fun

# Calcular reflexión y transmisión con epsilon optimizado
R_opt, T_opt = calcular_RT(e_opt_real, e_opt_imag)

#Calcular suma
suma = R_opt + T_opt

#Calcular absorcion
absorcion = 1 - R_opt - T_opt
absorcionVNA = 1 - linmag1 - linmag2
# Graficar R vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, R_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag1, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|R|')
plt.title('Coeficiente de Reflexión (|R|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
#plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar T vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, T_opt, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, linmag2, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('|T|')
plt.title('Coeficiente de Transmisión (|T|) vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)

```

```
plt.legend()

plt.show()

# Graficar Absorcion vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, absorcion, color='b', label='Teórico (Optimizado)')
plt.plot(freq1, absorcionVNA, color='r', label='VNA')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Absorción')
plt.title('Absorción vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

# Graficar Suma vs v
plt.figure(figsize=(12, 6))
plt.plot(v_val, suma, color='b', label='Teórico (Optimizado)')
plt.xlabel('Frecuencia (Hz)')
plt.ylabel('Suma')
plt.title('Suma vs Frecuencia (v)')
plt.xlim(1e9, 12e9)
plt.ylim(0, 1.5)
plt.grid(True)
plt.legend()

plt.show()

print(f"Valor optimizado de epsilon: {e_opt}")
print(f"Error optimizado: {opt_error}")
```